

QINGYAO LI

Email: ly890306 AT sjtu.edu.cn ◇ Homepage: simonlqy.github.io

Location: Shanghai, China

RESEARCH OVERVIEW

My research aims to make large language models **reason and make decisions** in complex tasks and real environments, using executable, verifiable environments as the key leverage: the same **verification signal** serves as the reward for reinforcement learning at training time and as the verifier for search at inference time. I focus on two problem domains, **code intelligence** and **general agents**, along two methodological axes: **reinforcement learning** (training time) and **test time scaling** (inference time).

I am a direct-entry Ph.D. student in Computer Science and Technology at Shanghai Jiao Tong University (**Zhiyuan Honorary Doctoral Program**), advised by Prof. Weinan Zhang and Prof. Yong Yu, with a joint-training period at Nanyang Technological University (advisor: Prof. Bo An). I have published **8 first-author / co-first-author papers** across **ICML, ICLR, EMNLP ($\times 2$), ACL, KDD, CIKM, and AAMAS**, plus a most recent first-author preprint (SkillGate, arXiv, Aug 2026). My work spans two problem domains; published work concentrates on the former, while my current focus is advancing the latter:

- **Code Intelligence (code generation, verification, and debugging).** Code tasks are natively executable and verifiable, making tests and execution feedback the central leverage. On one side I build stronger verifiers that are harder to fool, confronting reward hacking and “pseudo-correctness” — adversarial RL for training test generators (ATGen, ICLR 2026), adversarial test search (AdverMCTS, ICML 2026), and execution feedback-enhanced process reward models (CodePRM, ACL 2025 Findings). On the other side I use execution feedback to drive inference-time search and repair — thought-level search (RethinkMCTS) and natural-language-mediated debugging (NL-Debugging), both EMNLP 2025.
- **General Agent and Agentic RL (long-horizon decision making in real environments).** I am currently working on **world models** for code agents at Tencent Youtu Lab, refining action-consequence prediction from tool-returned observations down to repository state and product-level effects (test flips, render changes) so the agent can rehearse consequences before acting. Before that, at Xiaohongshu I trained Code / Terminal Agents with **GRPO** in real execution environments such as **SWE-bench and Terminal-Bench 2.0**, studying **selective skill invocation** and token-level **credit assignment** in long-horizon tasks (written up as the first-author preprint SkillGate). My earlier sequential decision-making work — hierarchical RL (GEHRL, CIKM 2023) and privileged knowledge state distillation (PKSD, KDD 2024) — is the pre-LLM methodological groundwork for this line.

On methods, I have first-hand experience in reinforcement learning (GRPO, and algorithms such as PPO / DAPO) spanning algorithm design through large-scale training systems (Megatron + SGLang + Ray), and in test time scaling (MCTS / PRM / Best-of-N) spanning search algorithms, PRM training, and complete evaluation pipelines. Current focus: Agentic RL and world models for code agents in real execution environments.

EDUCATION

Shanghai Jiao Tong University

2022.9 – 2027.6 (*expected*)

Ph.D. candidate in Computer Science and Technology (Zhiyuan Honorary Doctoral Program)

Advisors: Prof. Weinan Zhang, Prof. Yong Yu

GPA: 3.91/4.00

Nanyang Technological University

2025.9 – 2026.3

Ph.D. Exchange, College of Computing and Data Science

Advisor: Prof. Bo An

Xi'an Jiao Tong University

2016.9 – 2022.8

B.S. in Automation (Qian Xuesen Class); entered two years early via the Young Talent Class (2016 cohort)

GPA: 4.05/4.3 (Rank: 1/25)

RESEARCH INTERNSHIP

Tencent Youtu Lab

2026.7 – present

Research Intern · World Models for Code Agents

- Research on **world models** for code agents in real software engineering environments: most existing world-model work predicts only the next tool-returned observation, yet a tool reporting “edit succeeded” does not mean the task moved forward. I am building a **product-level world model** benchmark that extends prediction targets from observations to **repository state changes** (step patch / diff) and **product effects** (official test flips, front-end render changes), giving the agent the ability to mentally rehearse “what happens if I take this action” before acting — and providing intermediate supervision finer than a binary reward for agent training.
- Collected transition data with complete causal chains (action → observation → repository diff → test consequences → render consequences) on real **SWE-bench Multimodal** tasks in Docker execution environments; designed **deterministic visual oracles** for the render side: an LLM writes an executable render probe and predicate for each issue (false on base, true on gold), validated by a battery of machine quality gates and visual judges — porting SWE-bench’s FAIL_TO_PASS semantics intact onto rendered products, where the render itself is multimodal but prediction targets and scoring remain fully textual and exact-match scorable.
- Built an **agent-mode** evaluation framework: the model is restored into the exact read-only repository state right before an action, explores autonomously, and predicts the action’s consequences along three branches (observation, repository change, product effect), under both single-step and full-trajectory protocols (mentally simulating cumulative edits from the initial state). Systematic evaluation of frontier open-source models exposes a shared blind spot — models can hardly predict which actions truly flip tests (especially regressions they never expect to cause), while cumulative repository-change prediction best differentiates reasoning ability.

Xiaohongshu

2026.3 – 2026.7

Algorithm Research Intern · RL for Code / Terminal Agents

Mentors: Wenxiang Jiao, Yuan Lu

- Research on long-horizon Agentic RL for Code / Terminal Agents in real execution environments: using **GRPO** to train a small open-source LLM (9B) inside a real Docker agent runtime to invoke a large-scale skill library (about 600 skills) and complete long-horizon terminal tasks, evaluated on **5 benchmarks** including **SWE-bench**, **Terminal-Bench 2.0**, and SkillsBench.
- Built a unified Agentic RL training and evaluation system from scratch (**Megatron + SGLang + Ray**). The crux is keeping the reward signal clean — verifier engineering decides whether RL trains at all: designed an abort/refill mechanism separating infrastructure failures from task failures (preventing contamination of GRPO advantages), added image

pre-building, and rewrote the verifier for high-concurrency execution, cutting a single verification from **300s+ time-outs to seconds** and per-step dropped samples from ≈ 50 to **0**, which yielded the project's first healthy reward training curve.

- Tackled the core question of how a long-horizon agent learns **selective skill invocation** (the skill gate, i.e. tool / skill selection for general agents). Identified **credit entanglement** between selector and executor: task-outcome credit contaminates the learning of selection behavior (reading the right oracle but failing the task is wrongly penalized; reading a misleading skill yet succeeding by luck is wrongly rewarded), and selection tokens are heavily diluted over long trajectories. Designed **token-level credit separation** for GRPO: the selector advantage applies only to action tokens that determine skill identity (first oracle read positive; misreads and repeated reads receive relatively negative in-group advantage), while the task advantage applies to the remaining execution tokens and excludes all skill-read calls — so selection is accountable only for what was chosen, and execution only for how the task was done. Oracle identity is used solely to compute training-time advantages and is never visible to the model, which must discriminate among mixed candidates on its own.
- Systematic evaluation over a grid of **280 tasks** $\times$ **5 benchmarks** $\times$ **3 conditions** (oracle / no-skill / mixed-slate): under a mixed-slate setting of **1 oracle + 15 distractor skills** per task, against a task-reward-only GRPO control with identical initialization and evaluation, the oracle read rate rose from 54.3% to **83.9%** and the misleading misread rate fell from 69.6% to **21.8%**; the trained 9B model surpassed the 27B base under both oracle and no-skill conditions (**50.7% vs 41.4%** under oracle; **41.1% vs 37.1%** at best under no-skill). The work is written up as the first-author preprint SkillGate (arXiv, Aug 2026).
- Diagnosed training signals systematically with ablations and **gradient / NLL probes**, root-causing several classes of training collapse (e.g. violating the on-policy assumption, causing collapse after about 40 steps) to keep large-scale RL training stable.

Huawei Noah's Ark Lab

2024.7 – 2025.5

Research Intern · Code Intelligence: Verifiers and Execution Feedback Mentors: Wei Xia, Xinyi Dai, Ruiming Tang

- Independently drove code intelligence research around execution feedback and verification, producing **4 first-author / co-first-author papers** during the internship (ICLR 2026, ACL 2025 Findings, and two EMNLP 2025).
- The four works play complementary roles — test generation (ATGen), process reward modeling (CodePRM), execution feedback-driven search (RethinkMCTS), and execution-error-driven debugging (NL-Debugging) — together forming a complete experimental pipeline from data construction through PRM training to search evaluation.
- Received the **Huawei Excellent Intern** award.

PUBLICATIONS

9 first-author / co-first-author papers (incl. 1 arXiv preprint, plus 1 domain survey), organized by problem domain: Code Intelligence → General Agent and Sequential Decision Making → Other Work.

Module 1 · Code Intelligence (across the code generation stack) *Building verifiers + execution feedback-driven search and repair*

AdverMCTS: Dual-Agent Adversarial MCTS for Combating Pseudo-Correctness in Code Generation

ICML 2026 · First author [[Paper](#)]

The bottleneck of search is not search itself but a verification signal that is sparse and easy to overfit — “passing public tests yet failing hidden tests”, i.e. pseudo-correctness, is precisely reward hacking in its inference-time form. I design a dual-tree adversarial architecture of a Solver and an Attacker: the Attacker continuously generates targeted tests against logical disagreements within the code pool and injects them as hard constraints into the Solver’s search, making the verification environment progressively more hostile; an LLM-Arbitrator replaces majority voting, raising the valid test rate from 38% to **80%**. The method is training-free and applies directly to closed-source and very large models: pass@1 of DeepSeek-V3.2 (671B) on APPS improves from 49.7% to **73.3%**, and Qwen3-4B from 38.7% (thought-level MCTS baseline) to **49.7%**.

ATGen: Adversarial Reinforcement Learning for Test Case Generation

ICLR 2026 · First author [[Paper](#)]

Training a “verifier generator”: the test generator is trained with a dual-objective reward (IO accuracy and attack success rate, optimized by GRPO), while an adversarial code generator keeps producing bugs that escape the current tests; the two co-evolve into a dynamic difficulty curriculum that breaks through the fixed difficulty ceiling of static datasets. The resulting tests serve three roles — as a **verifier**: a 7B model reaches an attack rate of **36.99%** (UTGen-7B: 16.24%) and finds **60%** more bugs than GPT-4-turbo; as a **Best-of-N filter**: pass@1 improves from 30.67 to **35.00**, approaching human expert tests; and as an **RL reward for downstream models**: significantly better than other test sources, closing the loop from verifier to training signal.

CodePRM: Execution Feedback-enhanced Process Reward Model for Code Generation

ACL 2025 Findings · First author [[Paper](#)]

A process-level verifier. I find that the quality of a code thought step must be judged together with its derived code and execution result, and that text-only criteria are unreliable — execution feedback is a necessary ingredient of reliable process supervision. I collect (thought step, derived code, execution feedback) triples as supervision and train a PRM using pass rate as a continuous label rather than a binary 0/1; at inference the PRM acts as the process verifier in a Generate-Verify-Refine (GVR) pipeline, enabling dynamic error correction during search and consistently outperforming vanilla PRM baselines across generation budgets.

RethinkMCTS: Refining Erroneous Thoughts in Monte Carlo Tree Search for Code Generation

EMNLP 2025 · First author [[Paper](#)]

Execution feedback is used not merely to score but to directly rewrite the reasoning process: MCTS is lifted from code space into thought space, better matching the semantic reasoning strengths of LLMs; block-level execution feedback rewrites erroneous thoughts so that search continues along corrected paths, preventing errors from propagating down the path. On HumanEval, pass@1 improves from 70.12 to **89.02** (GPT-3.5-turbo) and from 87.20 to **94.51** (GPT-4o-mini), with token cost reduced by up to **35.7%** relative to Reflection-style methods.

NL-Debugging: Exploiting Natural Language as an Intermediate Representation for Code Debugging

EMNLP 2025 · Co-first author [[Paper](#)]

Answers the question of *which layer to repair* once a verification signal arrives: conventional debugging patches directly at the code level and easily settles for local patches that cannot resolve deep algorithmic flaws. This work reasons backward from execution errors to algorithm-level logical misunderstandings, rewrites the solution sketch in natural language, and then writes it back into code — repairing the *problem representation* rather than the *code symbols*, continuing the feedback-rewriting idea of RethinkMCTS. Ex-

periments show that natural-language-mediated debugging significantly outperforms code-level patching on complex algorithmic logic flaws.

Module 2 · General Agent and Sequential Decision Making

Skill selection and credit assignment — with pre-LLM

antecedents (2023–2024)

SkillGate: Training In-Policy Skill Selection in Long-Horizon Agents

arXiv 2026.8 · First author [Paper]

Skill libraries now hold thousands of instruction files, making *which skill to read* a decision the policy itself takes mid-episode — yet no existing signal trains it. We show the default remedy, outcome-rewarded RL over the candidate slate, cannot, and name the structural failure **selector credit starvation**: under a broadcast, sequence-level advantage, the few tokens naming the chosen skill carry a vanishing share of the loss, and the credit they inherit is increasingly wrong-signed as trajectories lengthen (a correct choice is punished whenever the execution after it fails); auditing a completed run’s own training artifacts confirms all three properties worsen monotonically with horizon. SkillGate removes the failure by construction, partitioning the token support into two **disjoint credit channels**: outcome credit reaches only execution tokens, while a separate action-local advantage reaches exactly the skill-naming tokens, positive only when a trajectory’s single read is the correct one. On **5 agentic benchmarks** with a 16-candidate slate, SkillGate lifts a 9B policy from 40.8% to **53.2%** trial success — well ahead of the identical budget spent on outcome reward alone — while cutting misleading-candidate exposure by **two thirds** with fewer reads. Code and models released.

PKSD: Privileged Knowledge State Distillation — Privileged Features at Training, Unprivileged Policy at Deployment

KDD 2024 · First author [Paper]

Environment hidden states are unobservable and differ across instances, which destabilizes RL training. I use the hidden state as a privileged feature to train a strong RL agent, then transfer it by knowledge distillation into an adapter that relies only on observable information, so the deployed policy retains adaptivity without requiring extra features — significantly improving the recommendation performance of 5 representative models.

GEHRL: Graph Enhanced Hierarchical Reinforcement Learning under Long-Horizon Sparse Rewards

CIKM 2023 · First author [Paper]

For long-horizon, sparse-reward goal-oriented planning: a high-level model decomposes the overall goal while a low-level model recommends concrete concepts under a test-driven intrinsic reward, with a knowledge graph encoding state and constraining feasible ordering; it achieves SOTA across 3 simulation benchmarks built from real data and rules. Its decomposition of “high level selects, low level executes” is an early form of the selector / executor credit assignment problem I study today.

Other Work

Recommendation × LLM, domain survey

SSRec: Learning Structure and Knowledge Aware Representation with LLMs for Concept Recommendation

AAMAS 2026 EA · First author [Paper]

Introduces human knowledge systems into concept recommendation by leveraging the world knowledge of LLMs as textual representations, with a knowledge-graph-based text adapter that removes representation anisotropy; it outperforms ID-Text, Text-only,

and ID-only baselines and yields recommendations better aligned with human cognitive structure.

Adapting Large Language Models for Education: Foundational Capabilities, Potentials, and Challenges

arXiv 2024 · First author [\[Paper\]](#)

Conceptualizes LLM problem solving in education as the integration of five core abilities — mathematical reasoning, writing proficiency, programming skills, logical reasoning, and knowledge-based question answering — and systematically summarizes the methodologies used to enhance them, along with the potentials and challenges ahead.

HONORS AND AWARDS

Outstanding Graduate of Xi'an Jiao Tong University	2022.7
International Excellence Award in Baidu Big Data Competition	2020.9
National Second Prize in Mathematical Modeling	2019.10
Qiu Changrong Second-Class Scholarship	2019.9

SERVICES

Conference Reviewer: ICML 2026, KDD 2024, CIKM 2023, AAAI 2023

PATENTS

Concept Recommendation System Integrating Human Knowledge Structures and Student Knowledge States (Under Application)	2024
A Wireless Charger for Pacemakers (Utility Model Patent)	2022.12